

AXI-Stream Decoder Architecture Specification

Оглавление

Введение	2
Информация об используемых протоколах.....	2
Параметры AXI Matrix Calculator.....	2
Назначение	2
Функциональное описание	2
Структурная схема.....	3
Описание входных и выходных сигналов дизайна.....	3
Функциональная схема.....	5
Алгоритм работы модуля	5
1. axis_tx — передатчик AXI Stream.....	5
2. axis_rx — приёмник AXI Stream.....	6
3. matrix_calc — главный управляющий автомат.....	7
4. mat_transpose — транспонирование матрицы	9
5. mat_addsub — сложение/вычитание матриц	10
6. mat_det — вычисление определителя	10
7. apb_csr — регистровый интерфейс APB	11
Карта регистров APB (CSR)	12
Программирование блока.....	13
Действия для сброса или настройки блока	13
Особенности программирования регистров	13
Особенности работы с данными.....	13

Введение

AXI Stream Matrix Calculator включает в себя 2 входных и 1 выходной порта AXI Stream, по которым проходят транзакции в соответствии с описанным протоколом.

Информация об используемых протоколах

1. AXI-Stream протокол (далее - AMBA AXI4-Stream)
2. APB подобный протокол (далее - AMBA APB3)

Параметры AXI Matrix Calculator

Название	Значение	Описание
N	4	Размерность квадратной матрицы
DATA_W	16	Ширина шины TDATA интерфейса AMBA AXI4-Stream

Назначение

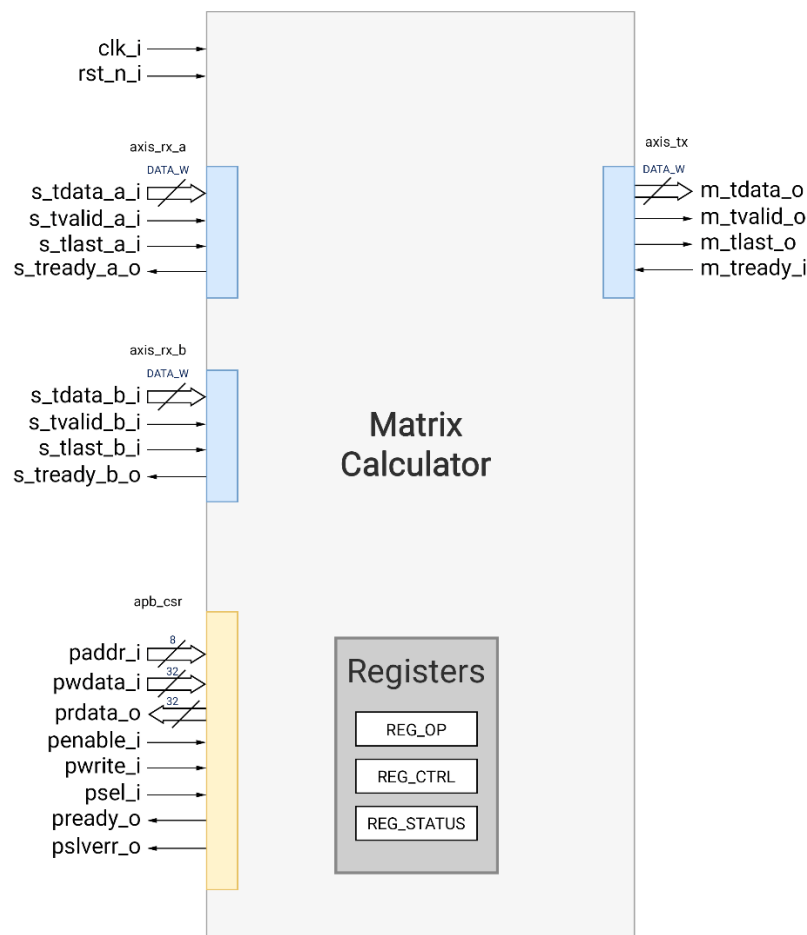
Модуль предназначен для выполнения базовых матричных операций над матрицами фиксированного размера с управлением через APB-интерфейс и обменом данными по AXI Stream.

Поддерживаемые операции: сложение матриц, вычитание матриц, транспонирование матрицы, вычисление определителя матрицы.

Функциональное описание

За передачу данных в Matrix Calculator и из него отвечают 2 входных и 1 выходной порты соответственно, работающие по протоколу AMBA AXI4-Stream. Доступ к управляющим регистрам дизайна осуществляется через выделенный порт, функционирующий по протоколу AMBA APB3.

Структурная схема

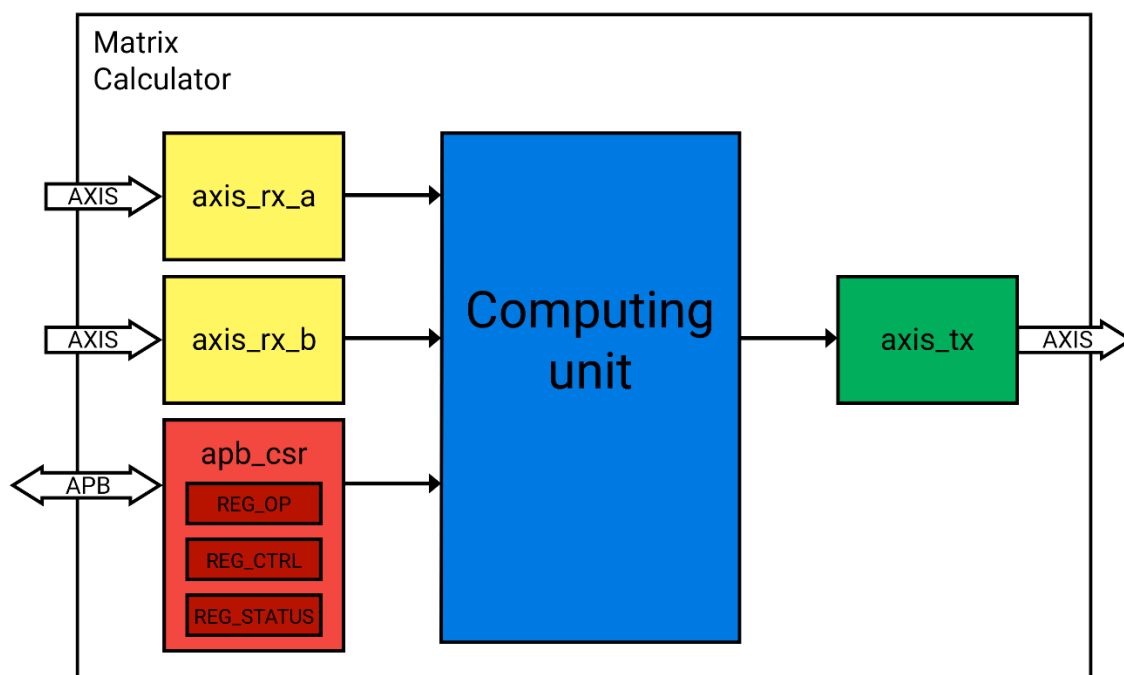


Описание входных и выходных сигналов дизайна

Название	Направление	Разрядность	Описание
clk_i	Input	1	Тактовый сигнал. Весь блок работает в одном тактовом домене
rst_n_i	Input	1	Синхронный сигнал сброса. Активный уровень – низкий
AXI Stream IN Входной интерфейс AXI Stream			
s_tdata_x_i	Input	DATA_W	Шина передачи данных
s_tvalid_x_i	Input	1	Сигнал валидности данных s_tdata_x_i на шине
s_tlast_x_i	Input	1	Сигнал последнего элемента матрицы в пакете

Название	Направление	Разрядность	Описание
s_tready_x_o	Output	1	Сигнал готовности slave к приёму данных s_tdata_x_i по шине
<i>AXI Stream OUT Выходные интерфейсы AXI Stream</i>			
m_tdata_o	Output	DATA_W	Шина передачи данных
m_tvalid_o	Output	1	Сигнал валидности данных m_tdata_o на шине
m_tlast_o	Output	1	Сигнал последнего элемента результата в пакете
m_tready_i	Input	1	Сигнал готовности slave к приёму данных m_tdata_o по шине
<i>APB Интерфейс для доступа к регистрам устройства</i>			
paddr_i	Input	8	Шина адреса
pwdata_i	Input	32	Шина данных от slave к master
prdata_o	Output	32	Шина данных от master к slave
penable_i	Input	1	Сигнал, маркирующий наличие активного запроса от master к slave
pwrite_i	Input	1	Сигнал, маркирующий тип запроса от master к slave (запись/чтение) 1 - запись 0 - чтение
psel_i	Input	1	Сигнал, маркирующий выбор slave устройства
pready_o	Output	1	Сигнал, маркирующий предоставление ответа на запрос от slave к master
pslverr_o	Output	1	Сигнал, маркирующий статус выполнения запроса 0 – выполнено успешно 1 – произошла ошибка

Функциональная схема



В состав модуля Matrix Calculator входят:

- **axis_rx** – блок, отвечающий за приём входных матриц по протоколу AXI4-Stream.
- **apb_csr** – блок, отвечающий за декодирование APB-транзакций, реализацию карты регистров управления и статуса.
- **Computing unit** – блок, отвечающий за выполнение арифметических и логических операций над матрицами.
- **axis_tx** – блок, отвечающий за выдачу результата вычислений по протоколу AXI4-Stream.
- Программируемые по APB-интерфейсу регистры - для управления всеми структурными блоками (**Operation, Control and Status Registers**).

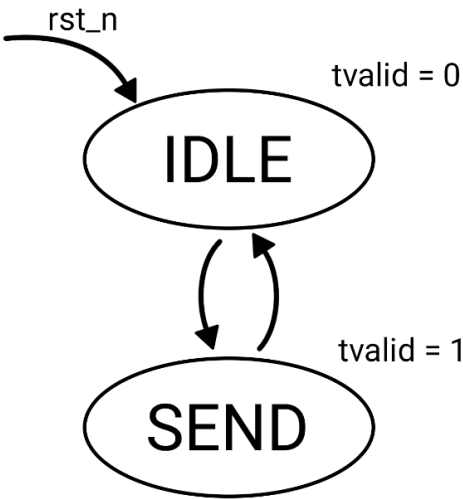
Алгоритм работы модуля

1. axis_tx – передатчик AXI Stream

Назначение

Модуль осуществляет последовательную передачу данных из внутреннего представления (матрица $N \times N$ или скаляр) в потоковый интерфейс AXI Stream. Поддерживает два режима: передача матрицы поэлементно и передача скалярного значения (результата вычисления определителя), разбитого на N слов.

Алгоритм работы FMS axis_tx



Состояния:

Состояние	Код	Описание	Изменения
IDLE	1'b0	Ожидание команды send	m_tvalid = 0
SEND	1'b1	Передача данных (матрица или скаляр)	m_tvalid = 1

Переходы:

IDLE → SEND по send
SEND → IDLE по (m_tvalid && m_tready && m_tlast)

2. axis_rx — приёмник AXI Stream

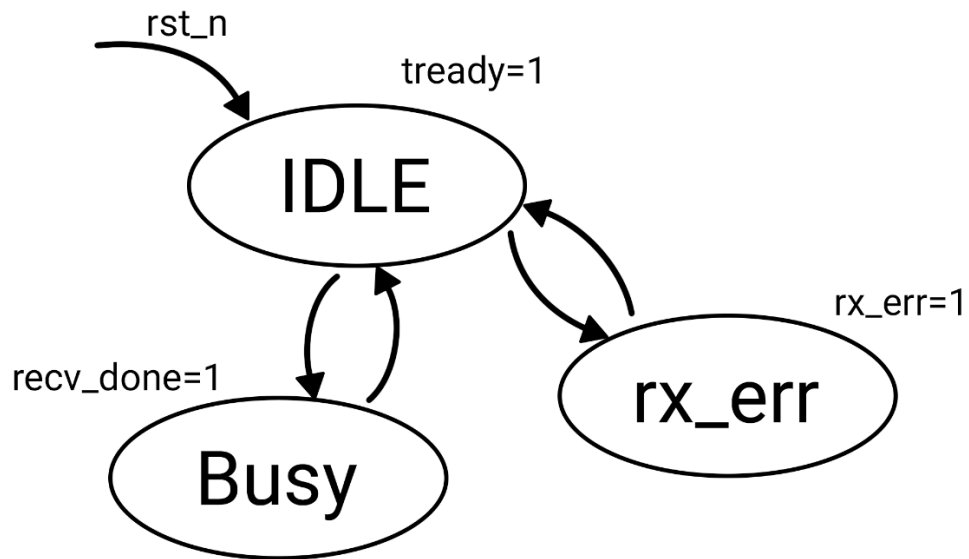
Назначение

Модуль принимает потоковые данные через интерфейс AXI Stream и сохраняет их во внутренний массив матрицы **N×N**. Осуществляет контроль целостности пакета (проверка длины) и выдачу сигналов об успешном приёме или ошибке.

Примечание:

Для операций, не требующих матрицы В канал s_axis_b игнорируется.

Алгоритм работы FMS axis_rx



Состояния:

Состояние	Код	Описание	Изменения
IDLE	2'b00	Ожидание начала пакета	s_tready = 1
BUSY	2'b01	Успешный приём полного пакета	recv_done = 1 s_tready = 0
RX_ERR	2'b10	Ошибка приёма (неверная длина пакета)	rx_err = 1 s_tready = 0

Переходы:

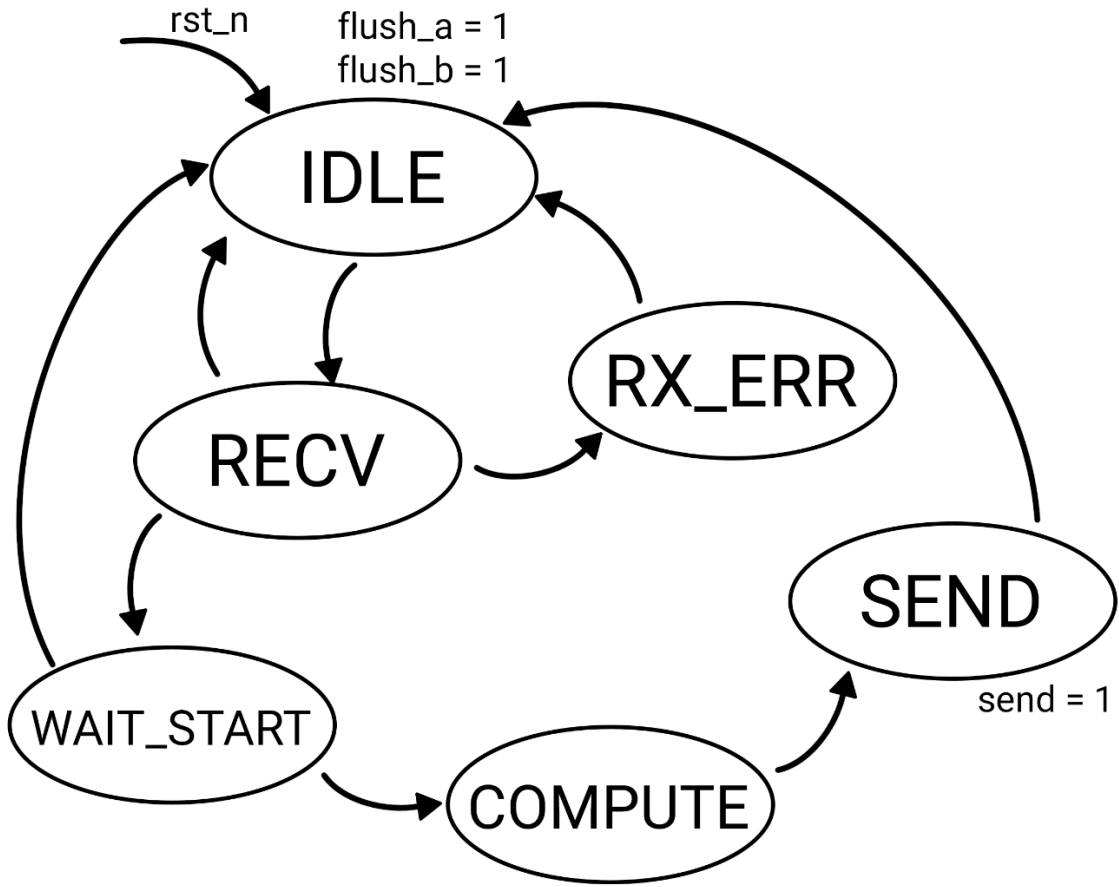
IDLE	→	BUSY	по (s_tvalid && s_tlast && (cnt == TOTAL-1))
IDLE	→	RX_ERR	по (s_tvalid && s_tlast && (cnt != TOTAL-1))
BUSY	→	IDLE	по flush
RX_ERR	→	IDLE	по flush

3. matrix_calc — главный управляющий автомат

Назначение

Модуль является центральным контроллером всего вычислительного блока. Он координирует работу всех подмодулей: приём данных через AXI Stream, выполнение выбранной операции (сложение, вычитание, транспонирование, определитель), передачу результата обратно в поток.

Алгоритм работы FMS matrix_calc



Состояния:

Состояние	Код	Описание	Изменения
IDLE	3'b000	Ожидание готовности приёмников.	flush_a=1 flush_b=1
RECV	3'b001	Приём матриц А и (при необходимости) В.	
WAIT_START	3'b010	Ожидание сигнала start от APB.	
COMPUTE	3'b011	Выполнение операции (add/sub/trans/det).	
SEND	3'b100	Передача результата через AXI Stream.	send = 1 (единичный импульс)
RX_ERR	3'b110	Ошибка при приёме данных.	

Переходы:

IDLE	→	RECV	no (s_axis_a_tready && s_axis_b_tready)
RECV	→	WAIT_START	no (recv_a_done && (op[1] recv_b_done))
RECV	→	RX_ERR	no rx_err_i
RECV	→	IDLE	no flush
WAIT_START	→	IDLE	no flush (приоритетнее start)
WAIT_START	→	COMPUTE	no start
COMPUTE	→	SEND	no (calc_done (op != 2'b11))
SEND	→	IDLE	no (m_axis_res_tvalid && m_axis_res_tready && m_axis_res_tlast)
RX_ERR	→	IDLE	no flush

Примечание:

При активации сигнала `flush` через APB-регистр `REG_CTRL[1]` оба входных AXI-Stream интерфейса переводятся в состояние `IDLE`, текущий приём данных прерывается, блок переходит в состояние `IDLE`. Таким образом возможна отмена текущей передачи или вывод блока из состояния ошибки. В состояниях `COMPUTE` и `SEND` запись `flush` игнорируется.

Если блок находится в состоянии `WAIT_START`, `COMPUTE` или `SEND` (`busy = 1`), попытка записи в поле `op` игнорируется. Значение `op` не изменяется, а текущая операция продолжает выполняться без изменений.

Так как к моменту приема данных тип операции установлен (предыдущее примечание), а также учитывая выражения для перехода в состояния `WAIT_START` и `RX_ERR`, канал B игнорируется для операций, требующих только матрицу A (игнорируется ожидание окончания приема (`recv_b_done`) и возникновение ошибки (`rx_err_b`))

4. mat_transpose — транспонирование матрицы

Назначение

Модуль выполняет операцию транспонирования квадратной матрицы размера `N×N`. Транспонирование — это зеркальное отражение матрицы относительно главной диагонали, при котором элемент `[r][c]` становится элементом `[c][r]`.

Алгоритм работы mat_transpose

Модуль реализован полностью комбинаторно с использованием `generate`. Для каждой строки `r` и каждого столбца `c` выходной матрицы формируется непрерывное присваивание:

```
mat_c[r][c] = mat_a[c][r]
```

5. mat_addsub — сложение/вычитание матриц

Назначение

Модуль выполняет поэлементное сложение или вычитание двух матриц A и B. Результатом является матрица C, где каждый элемент $C[r][c] = A[r][c] \pm B[r][c]$. Модуль также обнаруживает переполнение при арифметических операциях.

Алгоритм работы mat_addsub

Модуль реализован полностью комбинаторно с использованием generate. Для каждой пары элементов (r, c) выполняется следующая последовательность операций:

- 1) **Расширение разрядности:** входные значения (DATA_W) расширяются до (DATA_W+1) бит для временного хранения результата с учётом знака. Расширение выполняется путём копирования знакового бита в старший разряд.
- 2) **Выполнение операции:** в зависимости от сигнала sub:
Если sub = 0: выполняется сложение $result_ext = mat_a + mat_b$
Если sub = 1: выполняется вычитание $result_ext = mat_a - mat_b$
- 3) **Обнаружение переполнения:** переполнение возникает, когда результат операции не помещается в DATA_W бит со знаком. Для знаковых чисел переполнение определяется, когда знак результата (старший бит расширенного результата) отличается от знака, который должен быть у результата в усечённом виде. Логика обнаружения:
 $elem_overflow = result_ext[DATA_W] \wedge result_ext[DATA_W-1]$
- 4) **Формирование выходного значения:**
При переполнении: $mat_c = '1'$ (все биты установлены в 1) — насыщение до максимального значения
Без переполнения: $mat_c = result_ext[DATA_W-1:0]$ (усечение до DATA_W бит)
- 5) **Формирование общего флага переполнения:** $overflow = |elem_overflow|$ — логическое ИЛИ всех поэлементных флагов переполнения.

6. mat_det — вычисление определителя

Назначение

Модуль вычисляет определитель квадратной матрицы размера N×N методом Гаусса. Определитель вычисляется как произведение диагональных элементов после приведения, с учётом знака, меняющегося при перестановке строк. Модуль также сигнализирует о вырожденности матрицы.

Алгоритм работы mat_det

Модуль реализует алгоритм исключения Гаусса с частичным выбором ведущего элемента и целочисленной арифметикой. Работа выполняется за несколько тактов под управлением конечного автомата.

7. apb_csr — регистровый интерфейс APB

Назначение

Модуль реализует интерфейс APB (Advanced Peripheral Bus) для доступа к управляющим и статусным регистрам вычислительного блока. Обеспечивает связь между внешним процессором (через шину APB) и внутренними управляющими сигналами модуля matrix_calc.

Алгоритм работы apb_csr

Модуль работает как интерфейс между шиной APB и вычислительным блоком. Он хранит текущий код операции, транслирует управляющие импульсы (пуск и сброс) и предоставляет статусную информацию для чтения. Операции на шине APB выполняются без задержек (pready = 1), а ошибочные обращения блокируются сигналом pslverr.

Карта регистров APB (CSR)

Имя	Смещение	Доступ	Биты	Описание
REG_OP	0x00	RW	[1:0] OP	Код операции. По умолчанию 2'b00 (сложение)
		RO	[31:2] RESERVED	Чтение возвращает 0, запись игнорируется
REG_CTRL	0x04	RW	[0] START	Запись 1 — запуск вычисления. Самосбрасывается после старта
		RW	[1] FLUSH	Запись 1 — сброс состояния приёмника (например, при ошибке tlast). Самосбрасывается.
		RO	[31:2] RESERVED	Чтение возвращает 0, запись игнорируется
REG_STATUS	0x08	RO	[0] DONE	1 — результат доступен на m_axis_res, сбрасывается при новом START
		RO	[1] BUSY	1 — блок занят, входные каналы заблокированы (tready = 0)
		RO	[2] OVERFLOW	1 — переполнение в последней операции
		RO	[3] SINGULAR	1 — матрица вырождена (det = 0), только для OP=11
		RO	[4] RX_ERR	Ошибка приёма (несовпадение tlast и счётчика)
		RO	[31:5] RESERVED	Чтение возвращает 0, запись игнорируется

Программирование блока

Действия для сброса или настройки блока

Для корректной работы блока необходимо:

1. Подать тактовый сигнал `clk`;
2. Подать активный сигнал сброса `rst_n` (активный уровень – низкий);
3. Записать код выполняемой операции через APB-интерфейс;
4. Запустить выполнение операции, подав команду `start` через APB;
5. После получения результата пользователь может инициировать новую операцию и направить следующие данные для её выполнения.

При необходимости прерывания отправки текущих данных или вывода калькулятора из состояния ошибки:

1. Записать 1 в бит `flush` регистра `REG_CTRL` через APB;
2. Инициировать новую операцию и направить следующие данные для её выполнения;
3. Записать 1 в бит `start` регистра `REG_CTRL`.

При необходимости полного сброса состояния вычислений:

1. Подать активный `rst_n`;
2. Инициировать новую операцию и направить следующие данные для её выполнения.

Особенности программирования регистров

Все регистры блока после выхода из сброса (`rst_n = 0`) инициализируются нулевыми значениями

Общие правила получения сигнала ошибки (`pslverr`) на APB-интерфейсе:

1. При обращении по адресу за пределами адресного пространства (`0x00–0x08`) сигнал `pslverr` выставляется в высокий уровень;
2. При обращении по адресу `0x08` (RO-регистр) с операцией записи также выставляется `pslverr`;
3. При обращении к существующим регистрам с корректным адресом `pslverr` не выставляется.

Особенности работы с данными

1. **Матрицы A и B** передаются через 2 независимых AXI-Stream интерфейса (параллельно):
 - Размер матрицы: `N×N` элементов (по умолчанию 16-битных)
 - Передача идёт построчно: сначала `[0][0]`, `[0][1]`, ..., `[N-1][N-1]`
 - Сигнал `tlast` выставляется на последнем элементе (`cnt == N*N - 1`)

2. Для операции определителя:

- Требуется только матрица A
- Матрица B не используется (канал B игнорируется)
- Результат (скаляр) передаётся как единственный пакет из N элементов

3. Для сложения/вычитания:

- Требуются обе матрицы A и B
- Результат – матрица того же размера

4. Для транспонирования:

- Требуется только матрица A
- Результат – транспонированная матрица